A sign extension optimization pass for LLVM

Aggelos Gkogkosis
Supervisor: Polyvios Pratikakis

Department of Computer Science, University of Crete,
Heraklion, Crete, Greece

Abstract

Most programs running on modern 64-bit architectures, use many narrow types internally. In C/C++ for example, this is common for several reasons: many programs were originally designed for 32-bit architectures, programmers habitually reach for types like 32-bit "int" that are narrower than the register width, and some data naturally fits in fewer bits, such as characters. The compiler must insert sign and zero extension instructions to keep values correct at the wider register width. These extensions are frequent and often survive into the final generated code, where they contribute to unnecessary overhead.
This thesis studies sign and zero extension optimizations in LLVM, using the algorithm of Kawahito et al. and adapting it to its SSA-based IR (Intermediate Representation) for the RISC-V RV64 target. The implementation follows the main phases of the original approach, including widening, extension insertion, elimination order determination, and redundant extension elimination.
Our experimental implementation of the approach in LLVM IR shows promising results. On average, the transformation reduces the dynamic instruction count by approximately 0.24% on the LLVM Test Suite and 0.14% on MiBench. In some benchmarks, the reduction is more noticeable, ranging from around 0.5% to 1.15%. These results provide some interesting insights into the application of the Kawahito et al. extension elimination approach in LLVM and highlight its potential to reduce extension-related overhead in certain cases.

1 Introduction

When a program is compiled to the target architecture, values whose sizes are defined to be smaller than the architectural register size, must be adjusted to the register size. This is called sign extension. In C/C++, the use of narrow values remains common for several reasons. Many software systems were originally developed for 32-bit architectures and therefore retained their original data representations, programmers frequently rely on familiar types such as 32-bit ‘int‘ even on wider machines, and many kinds of data naturally require fewer bits, including characters, and small counters.

LLVM IR provides two bit-extension instructions: sign extension (`sext`) and zero extension (`zext`). The `sext` instruction extends the bit-width of a value while preserving its sign and numeric value, by replicating the most significant (sign) bit into all newly added higher-order bits. The `zext` instruction extends the bit-width of a value by

---

This thesis submitted in partial fulfillment of the requirements for the Bachelor of Science degree in Computer Science.

The work has been supported by the Foundation for Research and Technology - Hellas (FORTH), Institute of Computer Science (ICS).

padding all newly added higher-order bits with zeros, thus preserving the numeric value mostly for unsigned integers.

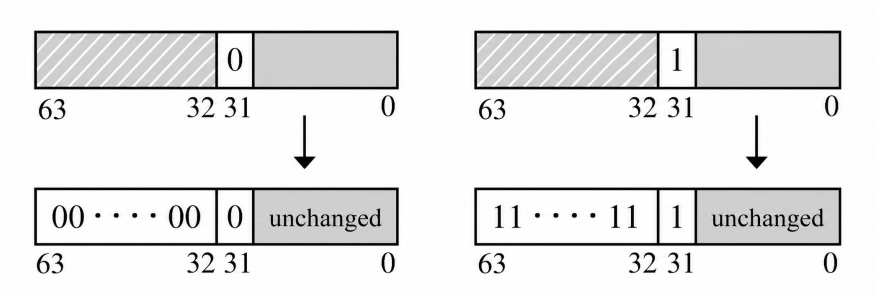


Figure 1. Sign extension of a 32-bit value to a 64-bit value.

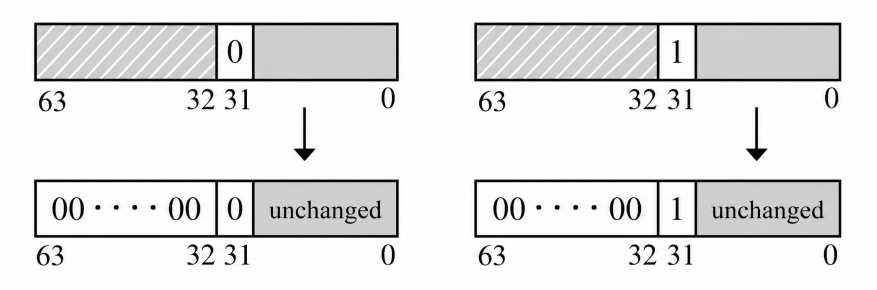


Figure 2. Zero extension of a 32-bit value to a 64-bit value.

In Figures 1 and 2, we see the two basic ways of widening a 32-bit value to 64 bits. Sign extension copies the sign bit into the new upper bits, while zero extension fills the new upper bits with zeros. Although both operations increase the bit-width of a value, they do not always preserve the same meaning. For example, the 32-bit value `0x80000000` (`-2147483648`) is negative as a signed 32-bit integer. If it is sign-extended to 64 bits, it becomes `0xFFFFFFFF80000000` and still represents `-2147483648`. If the same value is zero-extended instead, it becomes `0x0000000080000000`, which is interpreted as the positive 64-bit value `2147483648`. This is why the distinction between sign extension and zero extension becomes important during optimization and code generation.

In most architectures compiled through LLVM, register values are treated as zero-extended by default. As a result, middle-end optimizations tend to convert `sext` instructions into `zext`, and front-end authors are encouraged to prefer `zext` over `sext`. This means that by the time we see the IR, many extensions that were originally sign extensions have already been rewritten as zero extensions, even when this can no longer be proven from the IR alone. This becomes relevant on RISC-V RV64, where sign extension from 32 to 64 bits is often free, while zero extension is not [2, 4].

Motivated by the above, we adapt the algorithm of Kawahito et al. [1], proposed in 2002 for a Java Just-in-Time compiler targeting IA-64, to LLVM's SSA-based IR [3], targeting RISC-V RV64. The algorithm inserts extensions at definition points and then selectively eliminates them, starting from the most frequently executed regions of the program. In our implementation, the adaptation is organized around the following main ideas:

– To test the approach, we implemented two function passes for translating the program to 64-bit and effectively eliminating redundant extension instructions.

– We evaluated our results using LLVM's test suite and the MiBench benchmarks.

2 Implementation

Our implementation follows the structure of Kawahito et al. and adapts the algorithm to LLVM IR for RV64 as we metioned in the previous section. The optimization is performed in several stages, starting from the preparation of the IR and ending with the elimination of redundant extension instructions. The algorithm consists of four main phases:

– translating the program's intermediate representation to a 64-bit architecture by widening instructions to 64 bits;

– inserting extra sign extensions before every instruction that requires a sign-extended operand;

– determining the elimination order, starting from the most frequently executed regions;

– eliminating redundant sign extensions using UD/DU chains.

The following sections describe each phase of the implementation in more detail.

2.1 Widening

The first phase translates the intermediate representation of the program from a 32-bit architecture form to a 64-bit architecture form by widening 32-bit (or narrower) instructions to 64 bits. To preserve the correct semantics of each instruction, truncation and sign-extension operations are inserted when necessary. This is required because a widened 64-bit instruction does not always produce the same result as its 32-bit original value. By truncating the result back to 32 bits and then sign-extending it to 64 bits, the original 32-bit behavior is preserved while allowing the value to remain in the widened representation.

This transformation is essential for sign-extension elimination. In order to eliminate a sign-extension instruction, a widened version of its source value must be available so that its users can be rewritten. Furthermore, the widened representation is required by the order-determination phase, which is described in a later section.

Original program

```
1: x = sub  i32 a, b
2: y = add  i32 x, c
3: s = sext i32 y to i64
4: z = or   i64 s, -50
5: t = and  i64 s, 214
```

After widening

(before elimination)

```
 1: x.w   = sub   i64 a.w, b.w
 2: x.t   = trunc i64 x.w to i32
 3: x.s   = sext  i32 x.t to i64
 4: y.w   = add   i64 x.s, c.w
 5: y.t   = trunc i64 y.w to i32
 6: y.s   = sext  i32 y.t to i64
 7: y.s.t = trunc i64 y.s to i32
 8: s     = sext  i32 y.t to i64
 9: z     = or    i64 s, -50
10: t     = and   i64 s, 214
```

(after elimination)

```
1: x.w = sub i64 a.w, b.w
2: y.w = add i64 x.w, c.w
3: z   = or  i64 y.w, -50
4: t   = and i64 y.w, 214
```

Figure 3. Example of widening followed by sign-extension elimination.

2.2 Sign Extension Insertion

In this phase, we insert sign-extension instructions at use points to improve sign-extension elimination in frequently executed regions such as loops. After the original sign-extension instruction, we insert a truncation and a new sign extension before the first user in the same basic block. This additional sign-extension instruction acts as a helper, allowing the extension to be moved closer to its uses and increasing the chances of eliminating sign extensions that appear inside frequently executed basic blocks.

Sign-extension insertion

(a) Before insertion

```
hot_BB:
1: i = or   i64 j, k
2: x = add  i64 v, i
3: y = sub  i64 x, 255
4: s = sext i32 y to i64

cold_BB:
5: y = mul  i64 s, 155
6: z = or   i64 y, 400
```

(b) After insertion

```
hot_BB:
1: i     = or    i64 j, k
2: x     = add   i64 v, i
3: y     = sub   i64 x, 255
4: s     = sext  i32 y to i64

cold_BB:
5: s.t   = trunc i64 y to i32
6: s.t.s = sext  i32 s.t to i64
7: y     = mul   i64 s.t.s, 155
8: z     = or    i64 y, 400
```

Figure 4. Example of sign-extension insertion.

(b) Optimized result with insertion

```
1: i     = or    i64 j, k
2: x     = add   i64 v, i
3: y     = sub   i64 x, 255

cold_BB:
4: s.t.s = sext  i64 y to i64
5: y     = mul   i64 s.t.s, 155
6: z     = or    i64 y, 400
```

Figure 5. Optimized result with sign-extension insertion.

Figures 4 and 5 show how sign-extension insertion works. In this example, the users of the `sext` instruction ((**4a**) - **line 4**) are located in a less frequently executed (**colder**) basic block. For this reason, an additional `sext` is inserted in the cold block ((**4b**) - **line 6**), while the original value can be used in the most frequently execute (**hot**) path. As a result, the most frequently executed `sext` is removed from the hot basic block and moved to a colder one. Although the sign extension cannot be completely eliminated in this case, the optimization reduces its cost by keeping it only in the less frequently executed code.

2.3 Order Determination for Elimination

Sign extensions should be eliminated starting from the most frequently executed regions. Otherwise, eliminating sign extensions in less frequently executed basic blocks first may reduce the opportunities for eliminating sign extensions effectively.

In the example shown in Figure 4(b), if we eliminate first the `sext` instruction of the less frequently basic block (`cold_BB`), the `sext` of the most frequenty executed basic block (`hot_BB`) will still be required. In this case, even though one sign-extension instruction is eliminated, the optimization does not achieve the maximum benefit because the remaining `sext` is executed more frequently.

Before elimination(Widened)

```
cold_BB:
1: x     = add   i64 i , k
2: y     = sub   i64 x , 255
3: y.t   = trunc i64 y to i32
4: y.s   = sext  i32 y.t to i64

hot_BB:
5: y.s.t = trunc i64 y.s to i32
6: y.s.s = sext  i32 y.s.t to i64
7: m     = mul   i32 y.s.s, 455
```

Optimized Result 1

```
cold_BB:
1: x   = add   i64 i , k
2: y   = sub   i64 x , 255
3: y.t = trunc i64 y to i32
4: y.s = sext  i32 y.t to i64

hot_BB:
5: m   = mul   i32 y.s, 455
```

Optimized Result 2

```
cold_BB:
1: x     = add   i64 i , k
2: y     = sub   i64 x , 255

hot_BB:
3: y.s.t = trunc i64 y to i32
4: y.s.s = sext  i32 y.s.t to i64
5: m     = mul   i32 y.s.s, 45
```

Figure 6. Example requiring order determination

Another example is shown in Figure 6. There are two `sext` candidates for elimination(**lines 4 - 5**). Although only one of them can be eliminated, **Optimized Result 1** is clearly better than **Optimized Result 2**.

To achieve this, we sort the basic blocks from the most frequently executed to the least frequently executed, allowing us to eliminate sign-extension instructions in the correct order and prioritize the hottest regions.

2.4 Sign Extension Elimination

Sign extensions are analyzed and eliminated in order of execution frequency, starting from the most frequently executed regions, as described in Section 2.3.

Our elimination approach, based on the algorithm proposed by Kawahito et al., consists of two main algorithms: `EliminateOneExtend` and `AnalyzeUSE`.

```
1: EliminateOneExtend(EXT) {
2:   initialize flags for all instructions;
3:   required = FALSE;
4:
5:   /* use DU−chain */
6:   for (I in all instructions that use
7:         the destination operand of EXT) {
8:     required = AnalyzeUSE(EXT, I, TRUE);
9:     if (required) break;
10:  }
11:
12:  if (!required)
13:    eliminate EXT;
14: }
```

Figure 7. The EliminateOneExtend algorithm.

```
1: AnalyzeUSE(EXT, I) {
2:   if (a flag of USE for I
3:       has already been set)
4:     return FALSE;
5:
6:   set a flag of USE for I;
7:
8:   switch(I) {
9:   case 1:
10:     /* The source operand of I does not require
11:        type S */
12:     return FALSE;
13:
14:   case 2:
15:     /* Requirement of type S of source
16:        operand of I is equal to that
17:        of the destination operand of I*/
18:
19:     /* use DU−chain */
20:     for (J in all instructions that use
21:          the destination operand of I) {
22:       if (AnalyzeUSE(EXT, J))
23:         return TRUE;
24:     }
25:
26:     return FALSE;
27:   }
28:
29:   return TRUE;
30: }
```

Figure 8. The AnalyzeUSE algorithm.

The `EliminateOneExtend` function processes every `sext` and `zext` instruction in the program, including extensions narrower than 32 bits. For each extension instruction, it runs the `AnalyzeUSE` algorithm on all instructions that use the extension (DU-chain). If `AnalyzeUSE` returns `TRUE`, the extension is considered necessary for the instruction I and is not eliminated.

The `AnalyzeUSE` algorithm analyzes all users of the instruction that is being examined and handles two different cases. In the first case, we check if an instruction `I` does not require sign or zero extension information from its source operand. In this case, the algorithm returns `FALSE`, which means that this use path does not require the extension instruction that we are analyzing. In the second case, the algorithm cannot tell whether the source operand requires sign or zero extension. Therefore, it continues the analysis by running `AnalyzeUSE` again for all users of instruction `I` until it finds one (or not) in the path.

Instructions that can be handled by the second case are instructions where the higher bits do not affect the lower bits, such as `add`, `sub`, `and`, `or`, `xor`, `phi`, and `shl`. For these instructions, if the higher bits of their operands are not required, their behavior on the lower bits remains unchanged.

The first case includes instructions that can provide information about the higher bits of a value. Examples are instructions `'trunc'`, `'and'` with a positive constant operand, and `'or'` with an operand whose higher bits and sign bit are set to 1. Using this information, the algorithm can conclude that the higher bits are not needed. As a result, the extension instruction being analyzed is not required and can be eliminated because none of the instructions in its use path use the higher bits.

```
(a) Before elimination
1: x = add   i64 i, j
2: t = trunc i64 x to i32
3: s = sext  i32 t to i64
4: m = mul   i64 s, -100
5: v = and   i64 m, 255
6: z = or    i64 s, -55

(a) After elimination
1: x = add   i64 i, j
2: t = trunc i64 x to i32
3: s = sext  i32 t to i64
4: m = mul   i64 s, -100
5: v = and   i64 m, 255
6: z = or    i64 s, -55
```

```
(b) Before elimination
1: x = add   i64 i, j
2: t = trunc i64 x to i32
3: s = sext  i32 t to i64
4: m = sub   i64 s, -100
5: v = and   i64 m, 255
6: z = or    i64 s, -55

(b) After elimination
1: x = add   i64 i, j
2: m = sub   i64 x, -100
3: v = and   i64 m, 255
4: z = or    i64 x, -55
```

Figure 9. Example of AnalyzeUSE.

In Figure 9, we can see two different elimination cases. In **After Elimination (a)**, the `sext` instruction cannot be eliminated because its use path contains a `mul` instruction, where the higher bits may affect the result, even if the remaining use paths eventually reach `and`/`or` masks(case 1). Therefore, `AnalyzeUSE` stops the analysis and keeps the extension. In **After Elimination (b)**, the `sext` instruction follows a safe use path and can be eliminated. The `and` instruction with the positive constant value 255 guarantees that the higher bits are not needed, allowing the extension to be removed. Similarly, an `or` instruction with a negative constant operand, such as -55, guarantees that the higher bits are already set to the required sign-extension pattern and therefore do not require the original extension instruction.

2.5 Design

The implementation consists of two LLVM IR function passes [8, 9]. The first pass includes the widening and sign-extension insertion phases described in Sections 2.1 and 2.2. We first identify all paths that contain `sext` and `zext` instructions. Then we promote the relevant instructions to 64 bits and insert the required `trunc` and `sext` instructions to preserve the original program semantics. After widening, we perform sign-extension insertion as described in the previous section. If the users of an extension instruction are located in a different basic block, we insert an additional `sext` or `zext`

instruction at the use point. This allows the extension to be moved between basic blocks and increases the opportunities for elimination in frequently executed regions.

The second pass contains the remaining phases of the optimization. First, we sort all basic blocks according to their execution frequency, from the most frequently executed to the least frequently executed. Then we collect all `sext` and `zext` instructions into a list and run the `EliminateOneExtend` algorithm for each extension instruction in that order. This allows us to eliminate redundant extensions starting from the hottest regions of the program. Although we prioritize the elimination of `zext` instructions over `sext` instructions, because RISC-V sign extensions are often free as discussed in Section 1, the algorithm is able to eliminate both redundant `zext` and `sext` instructions.

3 Evaluation

For the evaluation of our approach, we use benchmarks from the LLVM Test suite and the MiBench suite. The metrics considered are code size and dynamic instruction count for the O3 and Oz optimization levels targeting the RISC-V RV64 architecture. We mainly focus on the O3 optimization level, since it applies aggressive optimizations with the goal of improving execution performance and is therefore the most relevant setting for evaluating our approach. The Oz results are also included to provide a comparison with an optimization level that focuses on reducing code size.

Initially, all benchmark programs are compiled for the RV64 target and the generated code size is measured. Then, for each benchmark, we collect the dynamic instruction count using Spike [5, 6], the official RISC-V ISA simulator. The programs are executed on Spike together with the Proxy Kernel (pk) [7], which provides a simple runtime environment and handles the system calls required by the benchmarks. The dynamic instruction count is used to evaluate the effect of our approach on the executed code and to compare it against the baseline compilation.

Benchmark	Original	Optimized	Pct Change
sqlite3	685754	684863	-0.12%
ALAC-decode	719583	719510	-0.01%
SPASS	557005	556370	-0.11%
ldecod	544915	544692	-0.04%
lencod	810885	810569	-0.039%
oggenc	494037	494002	-0.0071%
lua	779212	779228	+0.0021%
kimwitu++	2315055	2315495	+0.019%

Figure 10. Code size results for llvm-test-suite at Oz.

Benchmark	Original	Optimized	Pct Change
sqlite3	818506	818618	+0.013%
ALAC-decode	723854	723854	+0.00%
SPASS	622074	620326	-0.28%
ldecod	595796	595944	+0.024%
lencod	948404	948676	+0.028%
oggenc	511426	511390	-0.007%
lua	806083	806095	+0.0014%
kimwitu++	2394819	2394783	-0.015%

Figure 11. Code size results for llvm-test-suite at O3.

Benchmark	Original	Optimized	Pct Change
sqlite3	1191072934	1192664153	+0.13%
ALAC-decode	112174248	112174397	+0.0001%
SPASS	2995196	2993580	-0.054%
ldecod	1448596	1448624	+0.0019%
lencod	1778874	1779320	+0.025%
oggenc	3367611	3367749	+0.0041%
lua	49293168	49293252	+0.0002%
kimwitu++	88034614	87982278	-0.059%

Figure 12. Dynamic instruction counts for llvm-test-suite at Oz.

Benchmark	Original	Optimized	Pct Change
sqlite3	1056095684	1056095811	+0.000012%
ALAC-decode	78642830	78642692	-0.00017%
SPASS	2414285	2386430	-1.15%
ldecod	1038120	1033004	-0.49%
lencod	1034752	1033989	-0.073%
oggenc	2973349	2973522	+0.0058%
lua	47878251	47879061	+0.0016%
kimwitu++	83857308	83833604	-0.028%

Figure 13. Dynamic instruction counts for llvm-test-suite at O3.

The results for the LLVM Test Suite show that our transformation has a small impact on code size, while some changes can also be observed in the dynamic instruction counts.

Looking at the O3 results first, the code size remains very close to the original version for all benchmarks. The changes are generally small, with SPASS showing the largest reduction (-0.28%). For the remaining benchmarks, the differences are very close to zero. Overall, the transformation has a very limited impact on code size.

The dynamic instruction counts show a different picture. While some benchmarks remain almost unchanged, SPASS, ldecod and lencod show reductions in the number of executed instructions, with SPASS achieving a reduction of more than 1.1%. Across all benchmarks, the average reduction is approximately 0.22%. One reason for this is that some extension operations are moved away from frequently executed paths and into regions that are executed less often. Because of this, fewer extension-related instructions are executed at runtime, which helps reduce the overall dynamic instruction count.

For Oz, the changes remain small overall. Several benchmarks show slight reductions in code size, which is in line with the goal of Oz optimizations. On the other hand, the dynamic instruction counts remain very close to the original version, with only minor changes across the benchmarks. As a result, the impact of our transformation is less noticeable under Oz than under O3.

Benchmark	Original	Optimized	Pct Change
consumer-jpeg	621068	621096	+0.0045%
basicmath	514740	514740	+0.00%
madplay	742257	742509	+0.034%
toast	532244	532216	-0.0053%
dijkstra	520861	520861	+0.00%

Figure 14. Code size results for MiBench at Oz.

Benchmark	Original	Optimized	Pct Change
consumer-jpeg	652624	652640	+0.0024%
basicmath	515412	515412	+0.00%
madplay	779321	779625	+0.039%
toast	541477	541461	-0.0029%
dijkstra	521293	521293	+0.00%

Figure 15. Code size results for MiBench at O3.

Benchmark	Original	Optimized	Pct Change
consumer-jpeg	7106528	7107059	+0.0075%
basicmath	144186416	144186427	+0.00%
madplay	28826077	28717772	-0.37%
toast	18284169	18283679	-0.0027%
dijkstra	44742791	44742808	+0.00%

Figure 16. Dynamic instruction counts for MiBench at Oz.

Benchmark	Original	Optimized	Pct Change
consumer-jpeg	6760463	6760931	+0.0069%
basicmath	143167792	143167803	+0.0000077%
madplay	25646601	25469129	-0.692%
toast	16394788	16394712	-0.00046%
dijkstra	33874230	33874247	+0.00005%

Figure 17. Dynamic instruction counts for MiBench at O3.

The MiBench results show a similar behavior to the one observed in the LLVM Test Suite. The impact of our transformation on code size remains small, while the dynamic instruction count results provide some additional insight into its effect on the generated code.

For the O3 setting, code size remains very close to the original version for all benchmarks. The largest increase is observed in madplay (+0.039%), while toast shows a small reduction (-0.0029%). The remaining benchmarks are practically unchanged. Overall, the differences in code size are very small.

The dynamic instruction counts show a similar pattern. Most benchmarks remain almost unchanged, while madplay shows the largest reduction, reaching approximately 0.69%. This is particularly interesting because the same benchmark shows a very small increase in code size. This highlights one of the main ideas behind our implementation: a small change in code size does not necessarily translate to more instructions being executed at runtime. Across all benchmarks, the average reduction in dynamic instruction count is around 0.14%.

For the Oz setting, the changes remain small for both metrics. The code size results stay very close to the original version, with only minor increases or decreases across the benchmarks. This is expected, because as mentioned earlier, Oz primarily focuses on reducing the size of the generated code.

The dynamic instruction counts also remain close to the baseline for most benchmarks. The only noticeable change is observed in madplay, which shows a reduction of approximately 0.37%, while the remaining benchmarks are almost unchanged. Overall, the impact of our transformation under Oz is smaller than in O3, suggesting that the optimizations performed by Oz leave fewer opportunities for our approach to affect the final code.

4 Discussion

There are some limitations in the current implementation that affect the evaluation results. As discussed earlier, our approach performs a widening step from 32-bit to 64-bit values. This helps expose opportunities for extension elimination, but at the same time it can introduce additional extension instructions. Although many of these instructions are removed during the transformation, some of them remain in the generated code.

The main reason for this is that our implementation does not include the backward analysis phase of the original Kawahito et al. algorithm [1], called AnalyzeDEF. This phase is used to find additional extension elimination opportunities and would remove many of the extra trunc-sext instructions that remain after the widening step.

During the development of the pass, we observed that many extension elimination opportunities depend on this backward analysis. For this reason, implementing AnalyzeDEF would likely allow more redundant extension instructions to be removed. It could also create additional opportunities for moving extension operations to less frequently executed regions of the program.

Another possible improvement to the current implementation would be the use of LLVM's Demanded Bits Analysis [10]. This analysis identifies which bits of a value are actually needed by the instructions that use it and could help expose more extension elimination opportunities during the forward analysis phase. By providing more information about how values are used, it could allow the transformation to handle more cases and eliminate a larger number of redundant extension instructions.

5 Conclusion

As discussed in the previous section, our current implementation does not include the backward analysis phase (AnalyzeDEF) of the original Kawahito et al. algorithm. However, the evaluation results show that the approach can still provide useful results even without this part of the algorithm. Although the widening step introduces some extra instructions, the forward analysis phase is still able to eliminate many redundant extension operations. Combined with the movement of extension instructions to less frequently executed regions of the program, this helps reduce the dynamic instruction count of the generated code, even in cases where the code size does not show a similar reduction.

The evaluation results also highlight the potential of a more complete implementation. While the current implementation does not include all parts of the original Kawahito et al. algorithm, the observed reductions in dynamic instruction count show that the main ideas of the approach are already effective within LLVM IR. A more complete implementation could expose additional extension elimination opportunities and potentially lead to further improvements.

References

[1] M. Kawahito, H. Komatsu, and T. Nakatani, "Effective sign extension elimination," in *Proceedings of the ACM SIGPLAN 2002 Conference on Programming Language Design and Implementation (PLDI)*. Berlin, Germany: ACM, 2002, pp. 187–198, https://dl.acm.org/doi/pdf/10.1145/1111596.1111599.

[2] P. Karouzakis, "Sign extension optimizations inside llvm," MSc Thesis, University of Crete, 2025, https://elocus.lib.uoc.gr//dlib/9/1/2/attached-metadata-dlib-1753958098-770850-18684/MSc_Thesis_Karouzakis.pdf.

[3] LLVM Project, "LLVM Language Reference Manual," https://llvm.org/docs/LangRef.html, 2026.

[4] RISC-V International, "The RISC-V Instruction Set Manual, Volume I: Un-

privileged ISA," https://docs.riscv.org/reference/isa/unpriv/unpriv-index.html, 2026.

[5] Chipyard Project, "Spike," https://chipyard.readthedocs.io/en/latest/Software/Spike.html, 2026.

[6] RISC-V Software Collaboration, "Spike RISC-V ISA Simulator," https://github.com/riscv-software-src/riscv-isa-sim, 2026.

[7] RISC-V Proxy Kernel Project, "RISC-V Proxy Kernel," https://github.com/riscv-software-src/riscv-pk, 2026.

[8] LLVM Project, "The LLVM Compiler Infrastructure Project," https://llvm.org/, 2026.

[9] LLVM Documentation, "Writing an LLVM Pass," https://llvm.org/docs/WritingAnLLVMNewPMPass.html, 2026.

[10] LLVM Doxygen, "llvm::DemandedBits Class Reference," https://llvm.org/doxygen/classllvm_1_1DemandedBits.html, 2026.